

README — skills-and-capabilities/

Shared Plugin Registry, AST Code Review Graphs & Analytical Tooling

W5+H Subsystem Identity

- **WHO:** Invoked by both Mode A (Claude Code CLI via ECC) and Mode B (Prime Agent RLM loop).
- **WHAT:** Cross-repository plugins, PyGraphify AST code graphs, NotebookLM analytical pipelines, and Obsidian sync tools.
- **WHEN:** Triggered during code reviews, deep cross-document research queries, or scheduled vault hygiene sweeps.
- **WHERE:** [nova-xorpus/skills-and-capabilities/](#) (Federated Subsystem Root).
- **WHY:** Centralizes analytical power into a modular shared library so individual agent swarms do not rewrite code review or RAG tools.
- **HOW:** Exposes standardized CLI entry points, Python functions, and JSON schemas registered in [manifest.jsonl](#).

Internal Directory Topology

skills-and-capabilities/

— README.md	# This document (Shared library & capabilities index)
— manifest.jsonl	# Local cryptographic catalog of tools and graph modules
— code-review-graph/	# PyGraphify / Graphify AST Parsing Engine
— graph_builder.py	# Scans codebases and generates structural
dependency graphs	
— visual_topology_map.md	# Visual mapping of cross-module relationships

```

└─ notebook-imp/          # Deep Corpus Research Engine
|
|   └─ query_notebook.py  # Analytical multi-document querying over the vault
|
|   └─ source_packager.py  # Bundles markdown notes into curated analytical
sources
|
|
└─ obsidian-skills/      # Vault Integration & Hygiene
|
|   └─ graph_sync.py      # Syncs flat markdown files with Obsidian graph view
|
|   └─ markor_sweeper.py  # Detects unregistered assets to prevent context leaks
|
|
└─ early-trend-scraper/  # Autonomous Pre-Trend Research Daemon
    └─ daily_scraper.py   # Scheduled web scraper monitoring tech repositories
    └─ stack_analyzer.py  # Detects divergence between incoming web data and
the vault
    └─ trending_databank.jsonl # Raw JSONL telemetry tracking early tech trends

```

Beginner-Proof Implementation Rules

1. **Strict Modularity:** Every tool must be a standalone script that runs via CLI with clear argument flags (`--dir`, `--file`, `--output`).
2. **Deterministic Exit Codes:** All scripts must return exit code `0` on success and non-zero on failure so agent loops can detect errors.